

你的学校 / Your University

我的 XCPC 代码模板库

Printable Team Reference Document

Team
你的队名 / Your Team

Members
队员 A / 队员 B / 队员 C

目录 / Contents

1	基础 / Basic	2
1.1	比赛模板	2
2	数据结构 / Data Structure	2
2.1	并查集 / Disjoint Set Union	2
3	图论 / Graph	2
3.1	Dijkstra	2
4	数学 / Math	2
4.1	Extended Euclidean algorithm	2
5	检查表 / Checklist	2
5.1	赛前五分钟检查	2
5.2	常用常数	2

1 基础 / Basic

1.1 比赛模板

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using i64 = long long;
5 using u64 = unsigned long long;
6 using i128 = __int128_t;
7
8 template<class T>
9 bool chmin(T& a, const T& b) {
10     return b < a ? a = b, true : false;
11 }
12
13 template<class T>
14 bool chmax(T& a, const T& b) {
15     return a < b ? a = b, true : false;
16 }
17
18 void solve() {
19     // Write the per-test-case solution here.
20 }
21
22 int main() {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     int tests = 1;
27     // cin >> tests;
28     while (tests--) solve();
29     return 0;
30 }
31
```

提交前检查：整数范围、数组边界、多测清空、递归深度，以及输出格式。

```
10 |         return (x % mod + mod) % mod;
11 |     }
12 | }
```

5 检查表 / Checklist

5.1 赛前五分钟检查

- 确认图是否有向，是否允许重边和自环。
- 先估计上界，再选择整数类型。
- 测试最小输入、空结构和全部相等的情形。
- 最后一次提交前重新阅读输出要求。

5.2 常用常数

常数	数值
π	3.141592653589793
$\sqrt{2}$	1.414213562373095
$\ln 2$	0.693147180559945

2 数据结构 / Data Structure

2.1 并查集 / Disjoint Set Union

```
1 struct DSU {
2     vector<int> parent, size;
3
4     explicit DSU(int n) : parent(n), size(n, 1) {
5         iota(parent.begin(), parent.end(), 0);
6     }
7
8     int find(int x) {
9         while (x != parent[x]) {
10             x = parent[x] = parent[parent[x]];
11         }
12         return x;
13     }
14
15     bool unite(int a, int b) {
16         a = find(a), b = find(b);
17         if (a == b) return false;
18         if (size[a] < size[b]) swap(a, b);
19         parent[b] = a;
20         size[a] += size[b];
21         return true;
22     }
23
24     bool same(int a, int b) {
25         return find(a) == find(b);
26     }
27 };
28
```

3 图论 / Graph

3.1 Dijkstra

Complexity: $O((V + E) \log V)$ with a binary heap.

```
1 using P = pair<long long, int>;
2 const long long INF = numeric_limits<long long>::max() / 4;
3
4 vector<long long> dijkstra(
5     int source,
6     const vector<vector<pair<int, int>>>& graph
7 ) {
8     int n = (int) graph.size();
9     vector<long long> dist(n, INF);
10     priority_queue<P, vector<P>, greater<P>> pq;
11
12     dist[source] = 0;
13     pq.emplace(0, source);
14     while (!pq.empty()) {
15         auto [du, u] = pq.top();
16         pq.pop();
17         if (du != dist[u]) continue;
18
19         for (auto [v, weight] : graph[u]) {
20             if (dist[v] > du + weight) {
21                 dist[v] = du + weight;
22                 pq.emplace(dist[v], v);
23             }
24         }
25     }
26     return dist;
27 }
28
```

4 数学 / Math

4.1 Extended Euclidean algorithm

For $ax + by = \gcd(a, b)$, the function below returns (g, x, y) .

```
1 tuple<long long, long long, long long> exgcd(long long a, long
2 long b) {
3     if (b == 0) return {a, 1, 0};
4     auto [g, x, y] = exgcd(b, a % b);
5     return {g, y, x - a / b * y};
6 }
7
8 long long inverse(long long a, long long mod) {
9     auto [g, x, y] = exgcd(a, mod);
10    assert(g == 1);
11 }
```